T A R E Q O

18 — Migration W11 sur le Supabase de production (runbook)

Version 2.1.3+23 · 14 septembre 2026

TAREQO

Table des matières

Étape 0 — Prérequis 3

Étape 1 — Sauvegarde (obligatoire) 4

Étape 2 — Pré-vol (lecture seule, SQL Editor) 4

Étape 3 — Appliquer `supabase/upgrade.sql` 4

- Option A — psql via le pooler, UNE transaction (recommandé — convention du projet) 5
- Option B — SQL Editor du Dashboard 5

Étape 4 — Vérification post-migration (lecture seule) 5

Étape 5 — Idempotence (optionnelle, confirme que la réapplication ne duplique pas) 6

Étape 6 — Publier l'app W11 (IMPORTANT — fenêtre de compatibilité) 6

Rollback (seulement si nécessaire) 7

Checklist finale 7

> Objectif : appliquer la Partie 22 de supabase/upgrade.sql au projet de production (dotklbpqmlnhcdifiukd) — amène la base EXISTANTE à l'état W11 : lots de stock FIFO + suivi des péremptions et **ventes au poids/bascule**, avec TOUTES les quantités converties d'unités entières en MILLIÈMES (×1000).
>
> Ce que fait la Partie 22 (idempotente — mêmes conventions que les Parties 7/8) :
> 1. 22.1 — nouvelles colonnes sur products (code/PLU, venda_por_fracao, unit_label) et purchase_items.expiry_date (péremption optionnelle par article) ;
> 2. 22.2 — table stock_lots (un lot = un bon de réception : péremption, quantité initiale/actuelle, coût d'acquisition) + index FIFO ;
> 3. 22.3 — migration ×1000 de TOUTES les quantités (products, sale items et purchase items) + backfill des lots initiaux pour chaque produit avec du stock (l'invariant stock = Σ lots tient dès le premier jour) ;
> 4. 22.4 — RLS + politiques + triggers sur stock_lots ;
> 5. 22.5/22.6/22.7 — RPC recréés : process_sale avec **consommation FIFO des lots, `process_purchase` créant des lots** à la réception, get_sales_report avec des maths en millièmes.
>
> Le script est 100% idempotent et a été validé contre un vrai Postgres 16 (Docker, config CI exacte) : le sql_check.sh complet est vert (y compris réappliquant upgrade.sql deux fois sans double multiplication et la migration 25.50 → 2550 + 5 → 5000 sans effet de bord). Sans danger de le relancer autant de fois que nécessaire.
>
> Attention : IMPORTANT — à lire avant d'appliquer : la Partie 22.3 multiplie les quantités existantes par 1000 (5 unités → 5000 millièmes). Après cela, aucun client hérité (versions d'app pré-W11) ne doit écrire de données — la nouvelle version de l'app doit être publiée dans la même fenêtre (voir l'étape 6).
>
> Portugais (source faisant autorité) : `../pt/16-migracao-w11.md`

Étape 0 — Prérequis

- Accès au Dashboard du projet + mot de passe de la base

(Settings → Database → Connection string).

- L'hôte direct du projet est IPv6 uniquement → utiliser le pooler

aws-0-eu-north-1.pooler.supabase.com.

- psql local ou Docker (postgres:16 embarque psql).
- Attention : NE JAMAIS exécuter scripts/sql_check.sh contre la base live — il

fait DROP SCHEMA public CASCADE. La vérification live passe uniquement par scripts/sql_check_live.sh (lecture seule).

- App construite sur la version W11 prête à publier (voir l'étape 6).

Définir l'URL (remplacer SENHA) :

```
export
SUPABASE_DB_URL='postgresql://postgres.dotklbpqmlnhcdifiukd:SENHA@aws-0-eu-north-1.pooler.supabase.com:6543/
postgres?sslmode=require'
```

Étape 1 — Sauvegarde (obligatoire)

- Option rapide : Dashboard → Database → Backups → « Create a backup »

(ou activer le PITR).

- Option CLI (port 5432 *session* pooler, adapté à pg_dump) :

```
pg_dump
"postgresql://postgres.dotklbpqmlnhcdifiukd:SENHA@aws-0-eu-north-1.pooler.supabase.com:5432/postgres?sslmode=r
equire" \
 --schema=public -f backup_pre_w11.sql
```

Étape 2 — Pré-vol (lecture seule, SQL Editor)

2.1 W11 déjà appliqué ? Si la table stock_lots existe déjà, la migration a eu lieu → passer à l'étape 4 (et vérifier l'étape 6) :

```
SELECT to_regclass('public.stock_lots') AS stock_lots_already_exists;
-- Attendu : NULL (non appliqué) ou public.stock_lots (déjà appliqué).
```

2.2 Échantillon des quantités actuelles (pour confirmer le ×1000 ensuite — noter quelques valeurs) :

```
SELECT id, name, stock_quantity FROM public.products
ORDER BY created_at DESC LIMIT 5;
```

2.3 Comptages de lignes (pour confirmer ensuite que rien n'a disparu) :

```
SELECT 'products' t, count(*) FROM public.products UNION ALL
SELECT 'sales', count(*) FROM public.sales UNION ALL
SELECT 'sale_items', count(*) FROM public.sale_items UNION ALL
SELECT 'purchases', count(*) FROM public.purchases UNION ALL
SELECT 'purchase_items', count(*) FROM public.purchase_items;
```

Étape 3 — Appliquer supabase/upgrade.sql

Option A — psql via le pooler, UNE transaction (recommandé — convention du projet)

```
psql "$SUPABASE_DB_URL" -X -v ON_ERROR_STOP=1 -1 -f supabase/upgrade.sql
```

- -1 = transaction unique : si une quelconque partie échoue, tout

est annulé — aucun état partiel.

- Attendu : aucune erreur (la Partie 22 n'émet que des NOTICE « already

exists » sur les colonnes/tables déjà créées — normal, c'est l'idempotence
qui travaille).

- La migration ×1000 + le backfill ne s'exécutent **que si stock_lots est

vide** (garde de la Partie 22.3) — relancer le script est un no-op.

Option B — SQL Editor du Dashboard

1. Ouvrir le SQL Editor du projet.
2. Coller le contenu complet de supabase/upgrade.sql.
3. Exécuter. L'éditeur exécute le script en un seul lot ; en cas d'erreur il

s'arrête et affiche le message. Comme le script est idempotent, corriger et
relancer est sans danger.

Étape 4 — Vérification post-migration (lecture seule)

4.1 Exécuter le script de vérification live (lecture seule ; au moment de
W11 il faisait 31 contrôles — le script a depuis grandi à 35, les nouveaux
contrôles 32–35 de clôture de caisse (32–34 plus l'aperçu serveur 35)
s'ajoutant sans affecter ceux-ci) :

```
SUPABASE_DB_URL="$SUPABASE_DB_URL" bash scripts/sql_check_live.sh
```

- Attendu : VERIFICAÇÃO CONCLUÍDA — todos os checks passaram, y compris

les 7 nouveaux contrôles W11 (25–31) :

- 25 — colonnes products.code / venda_por_fracao / unit_label ;
- 26 — purchase_items.expiry_date ;
- 27 — table stock_lots + RLS ON + index FIFO ;
- 28 — process_sale avec consommation FIFO des lots en millièmes ;
- 29 — process_purchase créant des lots ;
- 30 — get_sales_report avec des maths en millièmes ;
- 31 — invariant `stock = Σ lots actifs` (compte combien de produits ont

un stock différent de la somme de leurs lots — attendu 0 juste après la migration).

4.2 Contrôle manuel (SQL Editor) :

```
-- 1) Quantités migrées en MILLIÈMES ? (5 unités → 5000)
SELECT id, name, stock_quantity FROM public.products
ORDER BY created_at DESC LIMIT 5;
-- Comparer avec l'échantillon de l'étape 2.2 : les valeurs doivent être ×1000.
-- 2) Invariant W11 : stock produit = Σ lots actifs ?
SELECT count(*) AS products_with_stock_out_of_lot_sum
FROM public.products p
WHERE p.stock_quantity > 0
 AND COALESCE((
 SELECT SUM(l.current_quantity) FROM public.stock_lots l
 WHERE l.product_id = p.id AND l.store_id = p.store_id
 AND l.current_quantity > 0
 ), 0) <> p.stock_quantity;
-- Attendu : 0.
-- 3) Lots backfillés (sans péremption — produits non périssables) ?
SELECT count(*) AS initial_lots, count(*) FILTER (WHERE expiry_date IS NULL) AS no_expiry
FROM public.stock_lots;
-- 4) RPC W11 en place ?
SELECT p.proname
FROM pg_proc p JOIN pg_namespace n ON n.oid = p.pronamespace
WHERE n.nspname = 'public'
 AND p.proname IN ('process_sale','process_purchase','get_sales_report')
ORDER BY 1;
```

4.3 Comptages de lignes — comparer avec l'étape 2.3 (rien ne doit manquer).

Étape 5 — Idempotence (optionnelle, confirme que la réapplication ne duplique pas)

Répéter l'étape 3, puis confirmer qu'il n'y a pas eu de double multiplication :

```
-- Un produit connu : ex. 5 unités doivent être 5000 (une seule fois)
SELECT id, name, stock_quantity FROM public.products
ORDER BY created_at DESC LIMIT 3;
-- Et le nombre de lots ne doit PAS avoir doublé :
SELECT count(*) FROM public.stock_lots;
```

Étape 6 — Publier l'app W11 (IMPORTANT — fenêtre de compatibilité)

La migration change le contrat des quantités (unités → millièmes) dans la base. Pour éviter de mélanger les échelles :

1. Publier la version W11 de l'app immédiatement après l'étape 3 — la

nouvelle app est la seule à lire/écrire les quantités correctement.

1. Les clients hérités encore ouverts peuvent échouer à écrire (quantité hors

échelle) — la migration est une courte fenêtre de maintenance : appliquer upgrade.sql et publier l'app dès que possible.

1. L'app W11 reste 100% compatible hors-ligne (le SQLite local se migre

lui-même via la migration W11 dans AppDatabase).

Rollback (seulement si nécessaire)

> Attention : Il n'y a pas de rollback automatique pour la Partie 22.3 (×1000 + lots) — restaurer via la sauvegarde (étape 1). Pour revenir en arrière : restaurer la base depuis backup_pre_w11.sql et publier l'ancienne version de l'app.

Checklist finale

#	Élément	Vérification
1	Sauvegarde effectuée	le fichier backup_pre_w11.sql existe
2	Pré-vol propre	stock_lots n'existe pas ; échantillon de quantités noté
3	upgrade.sql appliqué	aucune erreur (transaction unique)
4	sql_check_live.sh	tous les contrôles (incl. W11 25–31)
5	Invariant stock = Σ lots actifs	products_with_stock_out_of_lot_sum = 0
6	Réapplication sans effet	idempotence confirmée (étape 5)
7	App W11 publiée	aucun client hérité n'écrit dans la même fenêtre
8	App en mode Supabase	POS balance/poids et alertes de péremption fonctionnels